Gap Analysis — Implementation vs. Original Brief

Date: 2026-04-23 **Scope:** Compare what the original brief (the "build me an Ignition chatbot for the coater line" PDF + the architecture/data-model docs in `docs/`) asked for, against what is actually shipped in this repo today.

Grouped into three columns: **Done** ✓, **Stubbed** 🟡 (file exists, behaviour is a no-op or `NotImplementedError`), **Not started** ✗. Every placeholder the operator must fill in before go-live is listed in §3 with the exact file, line, current value, and how to source the real value.

---

1. Foundations (A-series)

ID	Asked for	Status	Where it lives
A1	Postgres 16 schema — 27 tables across 8 groups	✓	scripts/setup_database.sql
A2	pgvector extension + ivfflat index on `document_chunks.embedding`	✓	setup_database.sql §"Extensions" + §"Indexes"
A3	Reference data — failure_modes, equipment_taxonomy, prompt_versions	✓	scripts/seed_reference_data.sql
A4	Initial line memory + canned process facts	✓	service/scripts/seed_initial_data.py
A5	BM25 sparse index (GIN on `to_tsvector('english', chunk_text)`)	✓	setup_database.sql — `idx_chunks_bm25_gin`
A6	Materialized view `v_rca_precision_daily` for outcome closure tracking	✓	setup_database.sql — refresh wired into nightly loop in service/main.py
A7	Audit + feedback tables (message_feedback, user_corrections, outcome_linkages, audit_log)	✓	setup_database.sql §"Audit & Feedback"

2. Service capabilities (B-series)

ID	Asked for	Status	Where it lives / what's missing
B0	Deterministic tool layer (percentile_of, compare_to_distribution, nearest_historical_runs, detect_drift, defect_events_in_window, chunk_search)	✓	service/services/tools.py
B0.5	Tool-calling LLM loop with citation collection + token accounting	✓	service/services/llm.py `_run_tool_loop`
B1	Hybrid retrieval — vector + BM25 fused via RRF, FM/equipment filter, MMR diversification	✓	service/services/retrieval.py `retrieve_chunks_hybrid`

B2	Cross-encoder reranker over top-N hybrid candidates	◍	service/services/reranker.py — pass-through stub. **Needed:** `pip install sentence-transformers`, load `BAAI/bge-reranker-base`, score (query, chunk_text) pairs, reorder. Stub file has full instructions inline.
B3	Structure-aware chunker (preserves headings, tables, bullet lists)	✓	service/services/chunker.py
B4	Query rewriter (multi-query + step-back abstraction)	◍	covered by B0.5 tool loop; standalone rewriter not separated
B5	Hypothetical-document embedding (HyDE) for cold-start queries	◍	not implemented; relies on B1 hybrid being good enough
B6	Self-consistency / k-sample voting for high-stakes answers	◍	not implemented
B7	Multivariate anomaly detection over the curated tag block	✓	service/services/deviation.py (was renamed from `anomaly.py`)
B8	Two-step RCA reasoning chain (hypothesize → adjudicate) + 2 prompts	✓	service/services/rca.py + `config/prompts/rca_step{1,2}_v1.txt`
B9	Change ledger — what changed since baseline (recipe, crew, shift, equipment)	✓	service/services/change_ledger.py; crew/shift now wired through `RecipeContext`
B10	Outcome closure — 24h follow-up sweep + precision view refresh	✓	service/services/outcomes.py + nightly hook in main.py
B11	Active-learning loop (correction → embedding boost / chunk demotion)	◍	feedback API stores signals; the loop that *consumes* them and adjusts ranking lives in `retrieval.py` (bounded ±30%); the explicit "active learning trainer" job is not implemented
B12	Local vLLM provider as a swap-in for OpenAI	✓	service/services/llm.py `LocalVLLMClient`, registered in `provider_for(...)`
B13	Evaluation harness — replay golden cases, score citation P/R, FM accuracy	◍	service/eval/harness.py — three `NotImplementedError` stubs with full implementation notes
—	Symphony video capture adapter	◍	service/services/symphony_capture.py returns `extraction_status: "stub"`. Needs your Symphony API endpoint + auth.

3. Placeholders that MUST be replaced before go-live

Every entry tells you: **(a)** the file + line, **(b)** the current placeholder value, **(c)** what to replace it with, **(d)** where to find that value.

3.1 Service config — `service/config/settings.py`

Setting	Current	Replace with	Source
`api_key`	`"dev-key-change-me"`	a 32+ char random string	run `python -c "import secr` `print(secrets.token_urlsa` once, store in `.env` as `API_KEY=` Ignition's `ai.config.API_KEY`
`database_url` password segment	`change_me_in_production`	the real Postgres password you set when provisioning the DB	whoever stood up Postgres (DBA `docker-compose.yml`); store in `DATABASE_URL=postgresql+a` `chatbot:<REAL_PW>@<HOST>:` `ignition_chatbot`
`service_env`	`"development"`	`"production"` once you cut over	hand-edit `.env`
`local_llm_endpoint`	`""`	e.g. `http://vllm-host:8000/v1` if you're self-hosting	the host running your vLLM conta empty if you're using OpenAI

3.2 Secrets / `.env` (see `INSTALL.md` Part 2.1)

Variable	Placeholder in INSTALL.md	What it actually is
`OPENAI_API_KEY`	`sk-PASTE_YOUR_OPENAI_KEY`	from https://platform.openai.com/api-keys — create a key scoped only to the org/project running this service
`AZURE_OPENAI_*` (4 vars)	commented out	only set if you use Azure instead of public OpenAI; values come from your Azure OpenAI resource's "Keys and Endpoint" blade
`EMBEDDING_MODEL`	default fine	leave alone unless you know why

3.3 Ignition gateway — `ignition/perspective/gateway_wiring.py`

Line	Placeholder	What to fix
45–50	`line_id = self.session.custom.activeLineId or "coater1"`	If you have more than one line, populate the session custom prop `activeLineId` from your view's binding to the active line tag. If only Coater 1 exists, leave as is.
165–175	`line_id = tag.tagPath.split("/")[1]`	Replace with however your UDT path encodes line. E.g. if your tag path is `[default]Shaw/F0004/Coating/Coater1/Alarms/...`, `split("/")[3]` gives `"Coater1"`. Test once in the Ignition script console.

3.4 Ignition gateway — `ignition/scripts/config.py` (created from `INSTALL.md` Part 5)

Variable	Placeholder	Source
`AI_SERVICE_URL`	`http://<HOST_IP_OF_AI_SERVER>:8000`	the static IP / DNS of the box running the FastAPI container — `ipconfig` on Windows or whatever your network team assigned
`API_KEY`	`PASTE_THE_SAME_API_KEY_FROM_2.1_ENV`	the **same** value you put in `.env` `API_KEY=...` above
`IGNITION_BASE_URL`	`http://<YOUR_GATEWAY_IP>:8088`	your Ignition Gateway's URL — the one you log into for Designer
`LINE_ID`	`"coater1"`	leave as `coater1` for the pilot; expand later
`TAG_PROVIDER`	`"[UnifiedNamespace]"`	from Designer → Tag Browser → top-level provider name (in brackets)
`COATER1_ROOT`	`TAG_PROVIDER + "Shaw/F0004/Coating/Coater1"`	walk Tag Browser to the Coater1 UDT root and copy the path

3.5 Seed data — `service/scripts/seed_initial_data.py` (lines 50–95)

The seeded line memory uses example equipment IDs (`coater1_zone3`, `coater1`) and product styles (`Style-A`, `Style-B`).

- If your real equipment IDs match these → no change needed.
- If they differ → search-and-replace before running `python -m scripts.seed_initial_data`.
- The seeded process facts (zone counts, fpm ranges, calibration drift notes) are **plausible defaults**. Have a process engineer review the ~12 entries and edit the strings before seeding.

3.6 Reference data — `scripts/seed_reference_data.sql`

`failure_modes` table is pre-populated with codes like `delam_hotpull`, `sag`, `coating_weight_var`, etc. These are the codes the LLM will use in classification + RCA. If your plant uses different vocabulary (e.g. `MAR` vs `delam_hotpull`), edit the SQL file *before* running it, because changing them later requires a migration of every historical `failure_mode_classifications.fm_code` value.

---

4. What was asked for but is intentionally NOT built

- **Writing back to PLCs** — explicitly out of scope. Architecture principle #1 is "read-only".
- **Multi-line / multi-plant** — schema supports it (every table has a `line_id`), but seed data + Perspective view ship configured for Coater 1 only. Adding lines is a config exercise, not a code one.
- **Mobile / offline mode** — Perspective's responsive layout works on tablets; native mobile app and offline cache are not implemented.

- **Voice input** — not implemented.

5. Test coverage snapshot

- **148 tests passing, 2 skipped, 0 failing** as of this commit.
- Coverage is unit + integration with the DB layer mocked or in-memory.
- **Not yet exercised:** real Postgres + pgvector roundtrip, real OpenAI call, real Ignition gateway pairing, load testing.

6. Recommended go-live sequence

1. Replace every placeholder in §3.
2. `docker compose up postgres` → run `setup_database.sql` then `seed_reference_data.sql` then `python -m scripts.seed_initial_data`.
3. `docker compose up service` → hit `GET /api/health` → expect 200 with `{"db": "ok", "embeddings": "ok"}`.
4. From a Python REPL on the service host: `curl -X POST /api/chat` with a hand-built `CuratedContextPackage` and the API key — confirm a grounded response with citations.
5. Wire Ignition (`INSTALL.md` Part 5), open the Perspective ChatView, send "what is the current state of zone 3?" — expect a real answer.
6. Run for one shift in **shadow mode** (operators see answers but a second engineer reviews each one via `/api/feedback`).
7. Flip `outcome_closure_enabled=True` (already default), let one full 24-hour cycle pass, then check `v_rca_precision_daily` for your first precision number.
8. Decide on B2 reranker, B5 HyDE, B6 self-consistency, B11 active learning, B13 eval harness based on what the first week of real traffic shows.